

Software Test Plan (STP) - Railway Reservation System (RRS)

Project: Railway Reservation System (RRS)

Version: 1.0

Authors: Team 2 QA (Praveen, Praveen Balaji P, Prithvi S Paraddi, Priyaan G)

Date: 17-09-2026

Status: Draft

1. Introduction

Purpose: This document defines the test plan for the Railway Reservation System (RRS) v1.0. It outlines objectives, scope, strategy, resources, schedule, and responsibilities for testing the console/networked reservation platform prior to release.

Scope: Testing covers all customer-facing, counter-clerk, and administrative modules implemented in C/C++. This includes:

- User registration, authentication, masked password entry, and account logout.
- Multi-station train schedule search, intermediate timetable lookup, and distance calculation.
- Real-time seat inventory tracking across classes (1A, 2A, 3A, SL, CC) and quotas (General, Tatkal, Ladies).
- Multi-passenger reservation (1-6 seats), dynamic fare calculation, age concessions, berth allocation, and unique 10-digit PNR generation.
- Atomic ticket commitment using file record locking and multi-threaded mutexes.
- Ticket cancellation with dynamic slab-based refund deductions.
- Automated FIFO queue cascades promoting RAC passengers to CNF and WL passengers to RAC upon cancellation.
- Administrative master catalog management, coach quota configuration, tariff adjustments, passenger manifest charting, and daily station cash reconciliation.
- Cryptographic password hashing (salted SHA-256), buffer overflow boundary protections, and append-only audit logging.

References: RRS Software Requirements Specification (SRS) v1.0, IEEE 829-2008 Standard for Software and System Test Documentation, ISO C++17 Programming Language Standard, ISO/IEC/IEEE 29119 Software Testing Standards.

Definitions: RRS: Railway Reservation System, PNR: Passenger Name Record (Unique 10-digit transaction key), CNF: Confirmed Berth Allocation, RAC: Reservation Against Cancellation, WL: Waiting List, STP: Software Test Plan, RTM: Requirements Traceability Matrix, ASan: AddressSanitizer (LLVM memory error detector), TSan: ThreadSanitizer (LLVM data race detector), FIFO: First-In, First-Out Queuing Discipline, BVA / ECP: Boundary Value Analysis / Equivalence Class Partitioning

2. Test Items

The following modular subsystems and C/C++ components represent the test items subject to verification:

- Authentication & RBAC Module (AuthModule, TerminalUI, UserStore): User registration, credential verification, masked password entry, and role-based access enforcement.
- Train Route & Timetable Catalog Module (CatalogModule): Station graph search, timetable lookups, intermediate distance computations, and train queries.
- Seat Inventory & Quota Engine (InventoryEngine, BerthAllocator): Live multi-class berth quotas, Tatkal/Ladies quotas, and berth preference allocation.
- Booking & PNR Generator Module (BookingEngine, TariffModule, PNRGenerator, TransactionStore): Multi-passenger forms, dynamic fare tariff calculations, 10-digit PNR issuance, atomic file locking, and e-ticket exports.
- Cancellation & Queue Cascade Engine (CascadeEngine, BookingEngine): Cancellation deductions, atomic seat restoration, and automated FIFO waitlist cascade (RAC->CNF, WL->RAC).
- Administrative Master & Reporting Module (CatalogModule, ReportingModule, LedgerModule): Train route onboarding, quota adjustments, tariff matrix updates, passenger charting, and daily counter collection reconciliation.
- Persistence & File Integrity Subsystem (FileStore, AuditLogger): Crash-resilient binary file serialization, record-level locking, and rolling SHA-256 audit logging.

3. Features to be Tested

- Authentication & User Management: RRS-F-001 (SHA-256 Login), RRS-F-002 (Masked Password & 3-Attempt Lockout), RRS-F-003 (Passenger Registration).
- Train Schedule & Route Inquiries: RRS-F-004 (Source-Destination Schedule Search), RRS-F-005 (Intermediate Timetable & Distances), RRS-F-006 (Real-Time Multi-Class Availability).
- Ticket Reservation & PNR Generation: RRS-F-007 (Multi-Passenger Form Capture, 1-6 seats), RRS-F-008 (Dynamic Fare & Concessions), RRS-F-009 (Seat Hierarchy CNF->RAC->WL), RRS-F-010 (Berth Allocation Lower/Upper/Middle), RRS-F-011 (10-Digit PNR Issuance), RRS-F-012 (Atomic Concurrency Locking), RRS-F-013 (E-Ticket Text Generation).
- Cancellation & Waitlist Cascade: RRS-F-014 (Partial/Full Passenger Cancellation), RRS-F-015 (Cancellation Refund Slabs), RRS-F-016 (Automated FIFO Queue Cascade RAC->CNF, WL->RAC).
- Administrative Operations & Reporting: RRS-F-017 (Admin Train Route Onboarding), RRS-F-018 (Coach Quota Configuration), RRS-F-019 (Tariff Matrix Updates), RRS-F-020 (Final Chart Generation), RRS-F-021 (Daily Station Cash Reconciliation).
- Non-Functional Performance & Reliability: RRS-NF-001 (Search Latency <= 150 ms), RRS-NF-002 (Booking Latency <= 500 ms), RRS-NF-003 (99.95% Crash Durability), RRS-NF-004 (50 Concurrent Terminal Sessions), RRS-NF-005 (ISO C++17 Clean Compilation), RRS-NF-006 (Clerk Workflow <= 45s).
- Security & Integrity Protections: RRS-SR-001 (Salted SHA-256 Digests), RRS-SR-002 (RBAC Access Enforcements), RRS-SR-003 (Buffer Overflow Boundary Protections), RRS-SR-004 (Immutable Audit Logging), RRS-SR-005 (PNR 32-bit CRC Checksum Verification).

4. Features Not to be Tested

- Physical Railway Signaling & Interlocking: Hardware interlocking, track circuits, and electronic signals (handled by dedicated rail safety firmware).
- Commercial Bank Clearinghouse Internals: Card payment settlement networks and banking host servers (mocked via local sandbox test stubs).
- GPS Locomotive Positioning Subsystems: Real-time satellite positioning and hardware telemetry feeds.
- Operating System Kernel & Device Drivers: Kernel-level scheduling and display terminal driver implementations (assumed validated by OS vendor).

5. Test Approach/Strategy

A V-Model verification approach is followed, combining white-box unit tests with black-box integration, concurrency, performance, and security testing.

Levels

- Unit Testing: Module-level verification of algorithms, data structures, and validation helpers using Google Test (GTest) and custom assertion suites.
- Integration Testing: Verification of inter-module interfaces (e.g., BookingEngine invoking InventoryEngine, TransactionStore committing via FileStore).
- System Testing: End-to-end execution of complete user workflows via automated CLI test drivers and scripted terminal sessions.
- Acceptance Testing (UAT): User Acceptance Testing conducted against formal SRS acceptance criteria by student peers and faculty evaluators.

Types

- Functional Testing: Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) applied to age limits, quota bounds, station codes, and booking dates.
- Concurrency & Race Condition Testing: Multi-threaded stress testing simulating 50 concurrent booking clerk threads attempting to reserve the final available berths simultaneously.
- Memory Safety & Leak Profiling: Continuous profiling with Valgrind Memcheck and AddressSanitizer (ASan) to guarantee 0 memory leaks, 0 dangling pointers, and 0 buffer overruns.
- Performance Benchmarking: Throughput and latency benchmarking measuring search response times under 500+ trains and atomic booking transaction durations.
- Crash Recovery Testing: Injecting hard process termination (SIGKILL) during active binary file writes to verify write-ahead logging (WAL) recovery and data store non-corruption.
- Regression Testing: Automated re-execution of test suites upon each code fix to ensure zero regressions.

Entry Criteria

Stable C/C++ build compiling without errors under `-std=c++17 -Wall -Wextra -Wpedantic`; test environment initialized with pre-generated train catalog and user master files; test automation harness verified.

Exit Criteria

100% of planned test cases executed across all 21 FRs, 6 NFRs, and 5 SRs; 0 Blocker, 0 Critical, and 0 High-severity defects open; Valgrind reports 0 bytes lost; AddressSanitizer reports 0 memory violations; concurrency stress tests report 0 race conditions across 5,000 parallel requests; 100% RTM coverage verified.

5.1 Security Validation

- Salted SHA-256 Password Hashing: Binary file inspection and RAM core dump scanning to verify passwords are stored exclusively as 64-character SHA-256 hex digests with unique salts; plaintext passwords strictly absent.
- Terminal Input Masking: Verification of POSIX `termios.h` non-canonical input mode ensuring passwords are masked (*) and never echoed to stdout or logs.
- Account Lockout: Attempting 3 successive invalid logins to verify automated account lockout and security event logging.
- Role-Based Access Control (RBAC) Boundaries: Simulating unauthorized CLI menu choices to ensure Passengers cannot execute Clerk/Admin operations and Clerks cannot alter Train Master catalogs.
- Input Fuzzing & Buffer Overflow Testing: Injecting long strings (10,000+ characters) and format specifiers into station and passenger fields to verify safe truncation and rejection under AddressSanitizer.
- PNR Checksum Anti-Fraud Validation: Generating synthetic and corrupted 10-digit PNRs to verify that invalid CRC-32 verification checksums are rejected.
- Audit Log Immutability: Validating that every booking and cancellation creates a synchronous, append-only log record with rolling block checksums.

6. Test Environment

- Host Hardware: x86_64 Multi-Core Workstation (Intel Core i7/AMD Ryzen, 16 GB RAM, 512 GB NVMe SSD)
- Operating System: Ubuntu 22.04 LTS (POSIX Linux) & Microsoft Windows 11 (64-bit with MinGW)
- Compilers & Standards: GCC 11.4+ / Clang 14.0+ enforcing ISO C++17 (`-std=c++17 -Wall -Wextra -Wpedantic`)
- Test Frameworks & Profilers: Google Test (GTest v1.12+), CTest, Valgrind v3.18+, LLVM AddressSanitizer (ASan), ThreadSanitizer (TSan)
- Automation & Scripting: Python 3.10+ (stress test traffic generator, CLI test drivers), Bash shell scripts, Git

- Baseline Test Data: 500+ active trains with multi-hop station timetables, 10,000 mock passenger records, pre-seeded quota files

7. Test Schedule

Milestone	Target Date
Test case design	Week 1, Days 1-3
Environment setup	Week 1, Days 4-5
Unit & integration test execution	Week 2
System test execution	Week 3, Days 1-3
Performance, concurrency & security testing	Week 3, Days 4-5
Regression cycle	Week 4, Days 1-3
UAT	Week 4, Days 4-5

8. Test Deliverables

- Software Test Plan (STP): Formal test strategy, scope, schedule, and execution plan (this document).
- Test Case Catalog (Manual & Automated): Detailed test specifications covering pre-conditions, input datasets, execution steps, and expected outcomes.
- Automated Test Scripts & Concurrency Harness: Google Test source files and Python multi-threaded traffic simulation scripts.
- Synthetic Test Data Suites: Standardized binary and text test files representing trains, stations, users, and pre-seeded bookings.
- Test Execution Logs & Sanitizer Reports: Raw test logs, terminal recordings, and Valgrind/ASan trace outputs.
- Defect Tracking Reports & Triage Logs: Structured defect logs documenting issue descriptions, reproduction steps, severity, owner, and resolution status.
- Final Test Summary Report (TSR): Final evaluation report summarizing test execution metrics, RTM completion, and release readiness certification.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Praveen	Author test plan, establish QA milestones, oversee test execution, manage RTM traceability, coordinate final sign-off.

Role	Name	Responsibility
Test Engineer	Praveen Balaji P	Develop automated Google Test suites, execute memory leak audits, AddressSanitizer fuzzing, search/booking latency benchmarks, crash recovery tests
Developer	Prithvi S Paraddi	Support unit test stubbing, investigate logged defects, implement code fixes, participate in regression re-tests.
Product Owner	Priyaan G	Approve test results, sign off readiness

10. Risks and Mitigation

Identified Risk	Severity	Mitigation Strategy
Concurrency race conditions leading to double-booked berths during simultaneous clerk booking.	High / Medium	Implement rigorous POSIX thread stress testing (TSan); enforce atomic mutexes and OS file record locking (fcntl).
Memory leaks or undefined behavior in C/C++ string and binary file serialization routines.	High / Low	Integrate automated CI execution with Valgrind Memcheck and AddressSanitizer (-fsanitize=address) on every commit.
Data corruption in persistent binary records following simulated system crashes or power failure.	Medium / Low	Implement write-ahead logging (WAL) / atomic rename techniques; verify durability via crash injection scripts.
Test schedule slippage due to high volume of manual test verification across 21 FRs.	Medium / Medium	Prioritize risk-based testing; automate repetitive inquiry, booking, and cancellation scenarios using Python test scripts.

11. Assumptions & Dependencies

- POSIX-compliant terminal emulator and C/C++ standard library support are available on all target test machines.
- GCC v11+ / Clang v14+ supporting ISO C++17 and standard POSIX threading libraries (pthread) are pre-installed.
- Synthetic test data generators are completed prior to Phase 3 to provide consistent datasets for automated testing.
- Course evaluation environment provides terminal shell access and allows running background multi-threaded processes.

12. Suspension & Resumption Criteria

Suspension Criteria: Testing shall be suspended if a critical build failure prevents clean compilation under -Wall -Wextra; if an unhandled segmentation fault occurs in core modules (UserStore or CatalogModule) blocking >30% of test cases; or if persistent database files become unrecoverably corrupted.

Resumption Criteria: Testing resumes when the development team delivers a verified bug patch addressing the fatal blocker; test data stores are restored from baseline snapshots; and a sanity smoke test suite (TC-AUTH-01, TC-INQ-01, TC-BOOK-01) completes with 100% pass rate.

13. Test Case Management & Traceability

Req ID	Requirement Summary	Test Case ID	Test Type	Module Under Test	Expected Verification Result
RRS-F-001	User Authentication (SHA-256)	TC-AUTH-01	Functional / Security	AuthModule	Valid login grants role menu; invalid credentials rejected with error.
RRS-F-002	Password Masking & Lockout	TC-AUTH-02	Security / UI	TerminalUI	Password masked with '*'; account flagged LOCKED after 3rd failure.
RRS-F-003	Passenger Registration	TC-AUTH-03	Functional	UserStore	Persists valid user struct; duplicate username rejected.
RRS-F-004	Train Schedule Search	TC-INQ-01	Functional	CatalogModule	Returns matching routes between stations with departure/arrival times.
RRS-F-005	Intermediate Timetable View	TC-INQ-02	Functional	CatalogModule	Displays formatted route table with all intermediate halt stations.
RRS-F-006	Real-Time Seat Availability	TC-INQ-03	Functional	InventoryEngine	Outputs exact seat counts across 1A, 2A, 3A, SL, CC, and quotas.
RRS-F-007	Multi-Passenger Form Capture	TC-BOOK-01	Functional	BookingEngine	Validates 1-6 passenger forms; rejects out-of-bounds passenger counts.
RRS-F-008	Dynamic Tariff & Concessions	TC-BOOK-02	Functional / Calc	TariffModule	Computes fare; applies 40% senior discount and 50% child fare.
RRS-F-009	Seat Hierarchy (CNF/RAC/WL)	TC-BOOK-03	Functional / Logic	InventoryEngine	Allocates CNF until exhausted, then RAC, then WL, then REGRET.

Req ID	Requirement Summary	Test Case ID	Test Type	Module Under Test	Expected Verification Result
RRS-F-010	Berth Allocation Algorithm	TC-BOOK-04	Functional	BerthAllocator	Assigns specific coach & berth (Lower/Middle/Upper) based on preference.
RRS-F-011	10-Digit PNR Generation	TC-BOOK-05	Functional / Integrity	PNRGenerator	Generates unique 10-digit numeric key; collision rate is 0.00%.
RRS-F-012	Atomic Booking Transaction	TC-BOOK-06	Concurrency / Stress	TransactionStore	Locks record; exactly one clerk receives final berth; no double-booking.
RRS-F-013	E-Ticket Receipt Generation	TC-BOOK-07	Functional / Output	ReportingModule	Outputs console receipt and generates formatted file (PNR_<id>.txt).
RRS-F-014	Individual/Full Cancellation	TC-CANC-01	Functional	BookingEngine	Updates passenger status to CANCELLED and recalculates active ticket state.
RRS-F-015	Cancellation Deductions	TC-CANC-02	Functional / Financial	TariffModule	Applies slab deduction rules (>48h, 12-48h, <12h) and issues refund balance.
RRS-F-016	Automated Queue Cascade	TC-CANC-03	Functional / Queue	CascadeEngine	Atomically promotes first RAC passenger to CNF and first WL to RAC (FIFO).
RRS-F-017	Admin Route Onboarding	TC-ADM-01	Functional / Admin	CatalogModule	Adds new train route; persists to binary train master across restarts.
RRS-F-018	Coach & Quota Configuration	TC-ADM-02	Functional / Admin	InventoryEngine	Modifies coach capacity and quota allocations without data corruption.
RRS-F-019	Tariff & Pricing Updates	TC-ADM-03	Functional / Admin	TariffModule	Updates base fare rates; future bookings compute with updated tariffs.
RRS-F-020	Passenger Manifest Charting	TC-CHART-01	Functional / Output	ReportingModule	Generates finalized alphabetical and coach-wise passenger chart.
RRS-F-021	Daily Accounting Ledger	TC-REP-01	Functional / Financial	LedgerModule	Reconciles counter bookings, cancellations, and net cash receipts.

Req ID	Requirement Summary	Test Case ID	Test Type	Module Under Test	Expected Verification Result
RRS-NF-001	Search Latency <= 150 ms	TC-PERF-01	Performance	CatalogModule	95th percentile search latency <= 150 ms across 10k automated queries.
RRS-NF-002	Booking Latency <= 500 ms	TC-PERF-02	Performance	BookingEngine	Average booking transaction time <= 500 ms with disk sync enabled.
RRS-NF-003	99.95% Crash Durability	TC-REL-01	Reliability	FileStore	Zero corrupted records upon simulated hard kill (SIGKILL) during write.
RRS-NF-004	50 Concurrent Clerks	TC-CONC-01	Concurrency	TransactionStore	Zero double-allocations observed across 5,000 multi-threaded bookings.
RRS-NF-005	ISO C++17 Compilation	TC-PORT-01	Portability	CoreEngine	Compiles cleanly under GCC/Clang with zero warnings under -Wall -Wextra.
RRS-NF-006	Clerk Booking Time <= 45s	TC-UX-01	Usability	TerminalUI	Trained clerk executes end-to-end booking in under 45 seconds.
RRS-SR-001	Salted SHA-256 Passwords	TC-SEC-01	Security / Crypto	CryptoModule	All passwords verified as 64-char hex digests; plaintext strictly absent.
RRS-SR-002	Role-Based Access Control	TC-SEC-02	Security / RBAC	SecurityManager	Unauthorized role menu bypass attempts return 403 Access Denied.
RRS-SR-003	Buffer Overflow Protections	TC-SEC-03	Security / Robust	InputSanitizer	String overflow inputs safely truncated/rejected; ASan reports 0 violations.
RRS-SR-004	Immutable Audit Log	TC-SEC-04	Security / Audit	AuditLogger	All booking/cancellation events append synchronous log with rolling hash.
RRS-SR-005	PNR Checksum Verification	TC-SEC-05	Security / Integrity	PNRGenerator	Synthetically modified or forged 10-digit PNRs fail CRC-32 checksum.

14. Test Metrics & Reporting

Metrics Collected:

- Test Execution Rate: Percentage of planned test cases executed against the test catalog (Target: 100%).
- Test Pass Percentage: Percentage of executed test cases that pass without defect (Target: >= 98% on final run).
- Defect Density & Severity: Number of confirmed defects categorized by severity (Blocker, Critical, Major, Minor per 1,000 lines of code).
- Defect Aging & Turnaround: Average turnaround time from defect identification to developer resolution and re-verification.
- Requirements Coverage: Percentage of SRS requirements validated by at least one passing test case (Target: 100%).
- Memory Violation Count: Memory leaks and invalid pointer operations detected under AddressSanitizer and Valgrind (Target: 0).

Reports:

- Daily Execution Dashboard: Summary of test cases executed, passed, failed, and open defects sent to team members at end of each test cycle.
- Weekly Defect Triage Report: Formal review meeting to prioritize critical fixes and update the risk register.
- Final Test Summary Report (TSR): Comprehensive final report delivered upon test completion, including aggregate execution results, RTM sign-off, and release readiness certification.

15. Approvals

Project Role	Name & Identification	Signature / Verification Date
QA Lead	Praveen	
Development Lead	Prithvi S Paraddi	
Product Owner	Priyaan G	